Bengali License Plate Recognition: Unveiling Clarity with CNN and GFP-GAN

The paper, our re-run on dataset v1.1, why the plate reader fails on our plates, what GFP-GAN really buys, and what to show and say to the faculty

Prepared for MD. Rokib Hasan Oli (paper reader) · Supervisor: Dr. Md Mansur ul Mulk · 24 October 2026
Paper: N. Afrin, M. M. Hasan, M. F. E. Safin, K. R. Amin, M. Z. Haque, F. Ahmed and M. T. R. Shawon, *26th ICCIT*, Cox's Bazar, 2023; arXiv 2312.10701, 6 pages. Page numbers are arXiv PDF pages.
Code and data: github.com/mhtusher131/Bengali-License-Plate-Recognition_Unveling-Clarity-with-CNN-and-GFP-GAN (MIT; five TensorFlow Colab notebooks and `Data.zip`, no weights).
Our run: `paper08_kaggle.ipynb` on Tesla T4 x2, 22 Oct 2026, dataset **v1.1** (confirmed in `environment.json`), 3.5 h; full output in `After Train/results.zip`. Extra checks run locally on 24 Oct 2026 (sections 12-13).

The whole report on one page

The paper in three lines. To read blurry Bangladeshi plates, the authors first sharpen the plate image with GFP-GAN, then cut it into single symbols with classic OpenCV steps (Otsu threshold and contours), and classify each symbol with a small CNN (17 classes). They report 92.38 % symbol accuracy (p.4) and 88 % average string similarity on 30 plates (p.5). They never test the reader without GFP-GAN.

Is the v1.1 training OK? Yes, with one thing to understand. All 5 jobs ended with `code=0`, dataset version 1.1 is confirmed, and a local re-run of the plate test gives the Kaggle numbers to the last digit. The one oddity is real but harmless: the paper's CNN recipe (batch 1, no normalisation, 40 epochs) became **unstable after epoch 5** on our 48,000 symbol crops, so the "last epoch" score is only 36.7 %. The kit keeps the best-validation checkpoint (epoch 5: **93.1 %**), and every later step used that one. ResNet-50 was stopped by the time cap at epoch 25 of 30 after it had levelled off (best epoch 20).

Five findings. (1) **The symbol classifier is fine:** 93.1 % of 4,107 test symbols (paper: 92.38 % on 17 easy classes); 95.4 % on the 21 common tokens, but only 23.9 % on 39 rare district words and letters. (2) **The plate reader is not:** it reads 0.15 % of our 651 test plates exactly (1 plate), with 54.3 % mean similarity (paper: 88 %). (3) **Why:** the contour step finds only **52 %** of the characters and all of them on 8.6 % of plates; even with perfect human boxes the per-symbol CNN reads 59.5 % of plates, against 88.9 % for our paper #2 character detector on the same 467 plates (144 vs 7 plates, $p < 0.0001$). (4) **GFP-GAN:** +4.8 similarity points (significant, $p < 10^{-10}$) but no extra exact plate worth the name, worse at night, and 34x the reading time (205.6 vs 6.1 ms per plate). (5) **The paper's own numbers reproduce** on its own data with our code (ResNet-50 87.5 %, paper 87.50 %; CNN 93.4 % at the paper's 32x32, paper 92.38 %), but its split leaks: 48 of 303 test crops copy a training crop, and on the clean crops the CNN loses 2.2 points.

What to tell the faculty in one sentence. "We tested this paper on 651 real Bangladeshi plates: its symbol classifier is good, but cutting a plate into symbols with contours finds only half the characters, so it reads almost no plate exactly, and GFP-GAN adds 34 times the cost for a few similarity points, so our system keeps a character detector and drops restoration."

How this report is organised

Part	Sections	Read it when
A. Understand the paper	1-6: problem, terms, data, method, results, weaknesses	before you explain the paper
B. Our v1.1 experiment	7-14: what we ran, is it OK, results vs paper, symbols, plates, GFP-GAN, original check, findings	before you quote any number
C. Use it	15-21: research use, benefits, faculty slides, script, Q&A, next steps, GitHub	before the faculty meeting

PART A · UNDERSTAND THE PAPER

1. What problem does the paper solve?

Street cameras in Bangladesh give small, blurry, low-resolution plate images. Earlier Bangla plate readers were trained and tested on clean images, so they break on real camera footage (p.1-2). The paper asks: **if we first make the plate image clearer with a GAN, can a simple CNN read the symbols?** The title word "Unveiling Clarity" refers to this restoration step.

The paper's answer is a four-step pipeline: restore the plate with **GFP-GAN**, clean it with classic image processing, cut it into single symbols with **contours**, and classify each symbol with a **4-layer CNN**. A frozen **ResNet-50** is tried as a second classifier. There is **no plate detector**: the input is already a plate image.

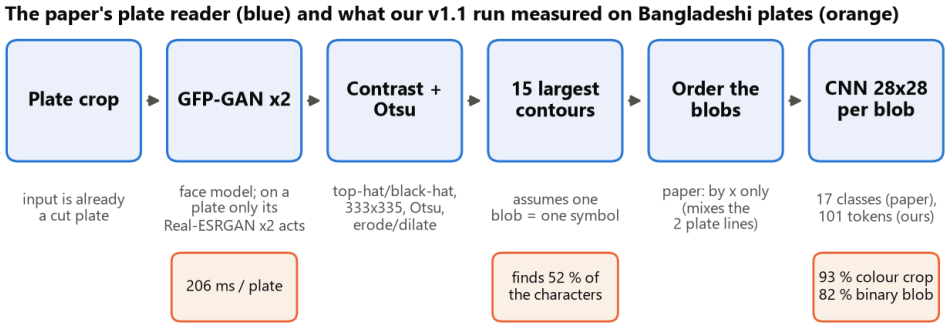


Figure 1. The paper's plate reader (p.3-5, and the authors' notebooks) with what our v1.1 run measured at each step (orange). Sections 11-12 explain the orange numbers.

2. Key terms in plain words

Term	Meaning
GFP-GAN	a GAN that restores *faces* using a face prior learned by StyleGAN (Wang et al., CVPR 2021). It finds faces first; on an image with no face it only runs its **background upsampler**
Real-ESRGAN x2	the background upsampler inside GFP-GAN v1.3: a super-resolution network that makes the image 2x bigger and sharper. On a plate, "GFP-GAN" = Real-ESRGAN x2
Top-hat / black-hat	morphology filters that boost bright and dark details; used here to raise contrast
Otsu threshold	picks one grey level that splits the image into black and white (binary image)
Contour	the outline of one connected black or white region; the paper assumes one region = one symbol
Segmentation	cutting the plate into one small image per symbol
Symbol / token	one unit on the plate: a digit, a series letter, "মেট্রো" (Metro) or a whole district word such as "ঢাকা"
SequenceMatcher similarity	Python difflib ratio = 2 x matching characters / (length of both strings). Partial credit: a plate with one wrong digit still scores about 0.9
Plate exact	the whole plate text is right; one wrong symbol = wrong plate. The strict number
CER / char accuracy	character error rate = edits needed / truth length; char accuracy = 1 - CER
Macro vs weighted F1	macro = plain mean over classes (a rare class counts as much as digit "1"); weighted = mean weighted by class size
Best-val vs last-epoch checkpoint	the model saved at the epoch with the best validation accuracy vs the model after the final epoch
McNemar test	compares two readers on the same plates using only the plates where exactly one is right; $p < 0.05$ = real difference
Wilcoxon signed-rank test	compares two paired lists of numbers (here: similarity of each plate with and without GFP-GAN)

3. The data

Item	Detail	Page
Symbol crops	2,024 crops, 17 classes: digits 0-9, series letters ক গ ঝ ল, region words ঢাকা, মেট্রো, নারায়ণগঞ্জ	p.3, Table I
Split	train 1,292 / val 429 / test 303 (the abstract calls the 1,292 "the dataset")	p.1, p.4
Source	cut by hand from a Kaggle synthetic plate set (furcifer) and the BLRPS set	p.2
Size	"standardized to 32x32"; in Data.zip 1729 of 2024 crops really are 32x32	p.3
Balance	digits 90-140 training crops each; letters and words only 13-30	p.3, Table I
Plate test	30 plates (the 30 plates are not released)	p.5

Their split leaks (our audit of Data.zip, repeated on 24 Oct). 48 of 303 test crops and 320 of 429 validation crops are copies of a training crop (grey 32x32, mean difference <= 1; 48 test and 320 validation crops are byte-identical files). **Every** test crop of ঢাকা (12), মেট্রো (10), নারায়ণগঞ্জ (10) and ঝ (14) is a copy, while the digits are clean (1 of 229). So the letter and word part of 92.38 % is partly memory, and the 98.8 % validation accuracy is inflated. Only 16 of 17 classes match Table I (class 9 is printed as 1 training crop; Data.zip has 100).

4. The method

Restoration and segmentation (p.3-4, notebooks GFPGAN.ipynb and Image_Processing.ipynb)

- GFP-GAN v1.3 with scale 2 and the Real-ESRGAN background upsampler. A plate has no face, so only the Real-ESRGAN x2 step changes the image (we checked the authors' notebook).
- Grey image, top-hat plus black-hat contrast, resize to 333x335, Otsu threshold, erode and dilate (with a tuple as the kernel, a small bug kept as is).
- The 15 largest contours become 15 symbol candidates, sorted left to right **by x only**, each pasted on a 24x44 white canvas and resized to 28x28 for the CNN.

The classifiers (p.2-5, notebooks CNN.ipynb and resnet50.ipynb)

	CNN	ResNet-50
Layers	4 conv (16/32/64/128 filters, kernels 22/16/8/4, "same"), max-pool 4, dense 512, dropout 0.4, dense 128, softmax	ImageNet ResNet-50, **all layers frozen**, Flatten, one Dense layer
Input	28x28 in the code (the text says 32x32), RGB / 255	150x150, RGB / 255, no ImageNet normalisation
Training	Adam lr 1e-4, **batch 1**, 40 epochs, small shifts	Adam (Keras default lr 1e-3; text says 1e-4), batch 1, 30 epochs, shear/zoom/flip
Parameters	3.71 M	28.7 M (5.17 M trainable)

Paper vs code (checked in the repository, commit 8dc72c7)

Topic	Paper	Authors' code
Input size	32x32x3 (p.3-4)	CNN 28x28, ResNet-50 150x150
ResNet-50 test accuracy	87.50 % (p.4)	the notebook prints 73.93 %
Plate score	average of 30 plates (p.5)	averages 10 hard-coded scores (87.56 %)
Plate demo	-	index-to-label list does not match the class order; crops not scaled by 1/255
Plate order	expected strings follow the x order (p.5, Table III)	x order mixes the two plate lines, so the "truth" is already mixed

5. What the paper reports

Result	Value	Page
CNN symbol test accuracy / precision / recall / F1	**92.38 %** / 0.9667 / 0.9162 / 0.9476	p.4, Table II
ResNet-50 test accuracy / precision / recall / F1	87.50 % / 0.9343 / 0.8827 / 0.8984	p.4, Table II
CNN validation accuracy / loss	98.80 % / 0.0303 (the same paragraph also says "92 %")	p.4
Whole plates: mean similarity	**88 %** on 30 plates; 17 of 30 >= 0.95, 24 of 30 >= 0.90	p.5
Example plate scores	1.00, 0.95, 0.86, 0.84	p.5, Table III
Comparison	proposed 88 %, ResNet-50 83 %, Kabiraj et al. (CNN + ESRGAN) 84 %, Haque et al. (ESRGAN) 78 %; different data, so not a fair race	p.6, Table IV
Speed, hardware, training time	not reported	-

6. Weaknesses (with page numbers, so you can show them)

- **No ablation of the main idea.** The title is about GFP-GAN, but no result without GFP-GAN is given (p.4-6). We measured it (section 12).
- **Leaky split** (section 3): 48 of 303 test crops and 320 of 429 validation crops copy training crops; all test crops of the 3 word classes and ঝ.
- **Only 17 symbols** (p.3): 3 of 64 districts and 4 of about 30 series letters. Most real plates contain a symbol the model cannot output.
- **x-only order** (p.5): Bangladeshi plates have two lines; sorting by x mixes them. Their expected strings use the same mixed order, so 88 % is easier than real reading.
- **Similarity, not exact match** (p.5): a mean SequenceMatcher ratio gives partial credit; the share of plates read fully right is not reported.
- **Tiny plate test**: 30 unreleased plates (p.5); no night, rain or real CCTV split.
- **GFP-GAN described wrongly** as a flow-based probabilistic model (p.2); it is a GAN with a face prior, and on plates its face part never runs.
- **Paper and code disagree** on input size, ResNet-50 accuracy and learning rate (section 4).
- **No speed numbers**, although GFP-GAN is the slowest step by far (section 12).

> **Fair credit.** It is one of the few Bangla plate papers with public code and labelled data, and it asks a good question (does restoration help?). That is why it is worth testing carefully rather than citing as proof.

7. What we ran

The notebook `paper08_kaggle.ipynb` ports the authors' Keras code to PyTorch and runs it on our **498R BanglaLPR All-in-One v1.1** data (all public Bangla sets merged, de-duplicated, grouped 70/15/15 split with seed 42, val/test with human labels only). Every labelled symbol box of the recognition set became one crop, in the authors' folder layout.

Job	GPU	What it does	Time
`cnn28`	0	paper CNN 28x28, 40 epochs, batch 1, Adam 1e-4, on our 101 tokens	152.5 min
`resnet50`	1	frozen ImageNet ResNet-50 + Dense, 150x150, 30 epochs planned, batch 1, Adam 1e-3, 3.2 h cap	199.5 min
`plates_cnn28`	0	the paper's plate pipeline on the 651 OCR test crops: GFP-GAN off/on x paper order/two-line order, plus perfect segmentation (human boxes, 467 crops)	3.0 min
`fps_cnn28 / fps_resnet50`	1	classifier speed, batch 1, alone on an idle T4	0.2 min each

Symbol crops	Train	Val	Test
crops (from plate crops)	47,992 (5,465)	4,643 (536)	4,107 (467)
classes present (of 101)	99	85	60

One test class, মুন্সিগঞ্জ (Munshiganj), has no training crop, so no model can get it right. The plate test uses the 651 human-typed OCR test crops (the same crops every team plate reader is scored on).

Differences from the paper: our data instead of theirs (101 tokens instead of 17, 37x more training crops); GFP-GAN is run as the network that actually acts on plates (Real-ESRGAN x2, via spandrel, FP16); a "two-line" order and a perfect segmentation test were added. Software: PyTorch 2.11 + CUDA 12.8, Python 3.13 (Kaggle, 22 Oct 2026 03:38).

8. Is the training OK?

Check	Result	OK?
Dataset version	`dataset_version: 1.1` in environment.json; folder BanglaLPR_AllInOne_v1_1; counts as expected	yes
Jobs	all 5 ended with code=0; 3.5 h of the 12 h Kaggle limit	yes
Data	47,992 training crops = exactly the number we count in the v1.1 labels	yes
CNN epochs	40 of 40; best validation at epoch 5 (90.2 %), then unstable (Figure 2)	yes, use best
ResNet-50 epochs	25 of 30: stopped by the 3.2 h time cap; best epoch 20, flat since about epoch 12	yes
Repeatable	a local re-run of the plate test with the run's best.pt (CPU) gives 49.49 % similarity without GFP-GAN and 59.53 % exact with human boxes: identical to Kaggle	yes
Code	the original-paper check with the same code gives about the paper's numbers on the paper's data (section 13)	yes

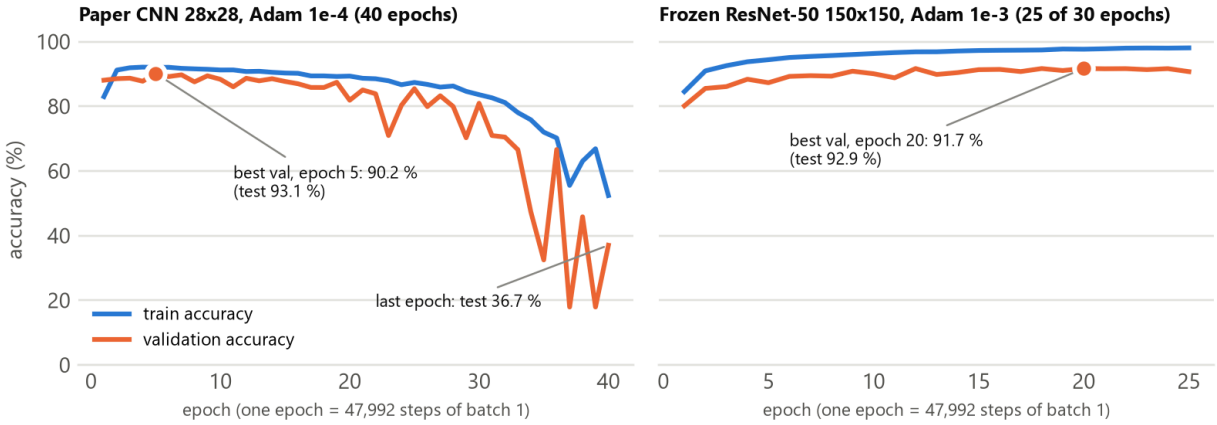


Figure 2. Accuracy per epoch. Left: the paper's CNN recipe reaches its best validation accuracy at epoch 5, then training accuracy itself falls (92 % to 52 %): the optimisation becomes unstable. Right: ResNet-50 (only the last layer trains) rises and stays flat.

Why did the CNN collapse, and is that a problem? The paper trains 40 epochs on 1,292 crops with batch 1, which is 51,680 weight updates in total. One epoch on our data is 47,992 updates, so our epoch 1 already equals the paper's whole training, and epoch 40 is 37 times more. Batch 1 with Adam, no batch normalisation, no learning-rate decay and very large kernels (22x22 on a 28x28 image) slowly drifts: the training loss rises from 0.37 (epoch 5) to 2.28 (epoch 40) and training accuracy falls too, so this is not over-fitting but instability. **It is not a bug in the run:** the kit saves the best-validation checkpoint, and the plate test and speed test used that model. The "last epoch" row in the kit's table (36.7 %) only shows that the paper's recipe does not scale; do not quote it as our accuracy.

9. Results next to the paper

Metric	Paper (page)	Ours: CNN	Ours: ResNet-50
Classes	17 (p.3)	101	101
Symbol test accuracy, best-val checkpoint	-	**93.08**	**92.89**
Symbol test accuracy, last epoch	92.38 / 87.50 (p.4)	36.67 (unstable)	92.06
Macro F1, best-val (all classes in test)	94.76 / 89.84 (p.4)	36.42	43.90
Macro F1, 21 tokens with >= 20 test crops	-	89.06	89.27
Weighted F1, best-val	-	92.31	92.75
Best validation accuracy	98.80 (p.4)	90.20	91.71
Classifier speed, batch 1, T4	not reported	760.9 FPS (1.31 ms)	171.5 FPS (5.83 ms)
Training time	not reported	151.8 min	198.2 min (25 epochs)

Whole plates (CNN, 651 test plates unless noted)	Paper (page)	Ours
Mean similarity, GFP-GAN + x order (the paper's exact recipe)	88 % on 30 plates (p.5)	44.95 %
Plates with similarity >= 0.95 / >= 0.90	57 % / 80 % (p.5)	0.00 % / 0.00 %
Mean similarity WITHOUT GFP-GAN, x order	never tested	41.86 %
Mean similarity, GFP-GAN + two-line order	-	54.30 %
Plate exact, GFP-GAN + two-line order	not reported	**0.15 %** (1 plate)
Plate exact, no GFP-GAN + two-line order	-	0.00 %
Char accuracy (1 - CER), GFP-GAN + two-line order	-	33.96 %
Perfect segmentation (human boxes, 467 plates): exact / char acc.	-	59.53 % / 86.77 %
Time per plate: GFP-GAN / segmentation + CNN	not reported	205.6 ms / 6.1 ms

> **How to read the comparison.** The paper used its own 17 easy classes and a leaky test; we use 101 tokens on a grouped, duplicate-free split. So the classifier rows show direction and size, not a race. The plate rows are different in kind: the paper scores 30 synthetic-looking plates with a forgiving similarity; we score 651 real plates and also report the strict exact match. The comparison on the paper's *own* data is section 13.

10. The symbol classifier: good on common tokens, weak on rare ones

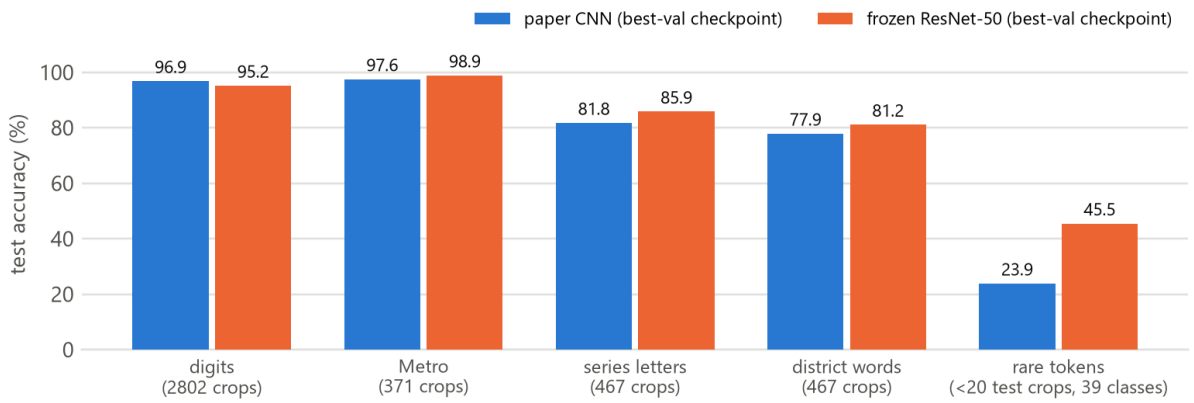


Figure 3. Test accuracy per token group (best-validation checkpoints). Both models read digits and মেট্রো well; letters and district words are harder; the 39 rare tokens (1-19 test crops each) are where the CNN fails and the pretrained ResNet-50 helps.

- **Digits carry the score:** 2,802 of the 4,107 test crops are digits (68 %), read 96.9 % right by the CNN.
- **Why macro F1 is only 36.4 %:** 60 classes appear in the test set; 39 of them have 1-19 crops and the CNN never gets 33 of these right. On the 21 tokens with >= 20 test crops (3,973 crops, 97 % of the test) macro F1 is 89.1 %.
- **ResNet-50 vs CNN:** about equal overall (92.9 vs 93.1 %), but ResNet-50 is better on letters (+4.1), words (+3.2) and rare tokens (45.5 vs 23.9 %), because its ImageNet features need fewer examples. The CNN is 4.4x faster (760.9 vs 171.5 FPS).
- **Typical CNN mistakes** (count): খুলনা read as ঢাকা (8); ৯ read as ১ (6); ০ read as ৩ (6); যশোর read as ঢাকা (6); ট read as ৮ (6); ৭ read as মেট্রো (5). Rare district words fall back to the most common word, ঢাকা; similar Bangla digits (৯/১, ০/৩) and ট/৮ are mixed up.

Compared with the paper: 93 % on 101 tokens with a clean split is at least as good as 92.38 % on 17 tokens with a leaky test. The per-symbol CNN idea is not the weak part of the paper. The weak part comes next.

11. Whole plates: where the reader breaks

The plate reader is only as good as its weakest step. We split the loss into its parts on the 467 test plates that have human symbol boxes (4,107 symbols), using the run's CNN locally (`report_src/seg_check.py`).

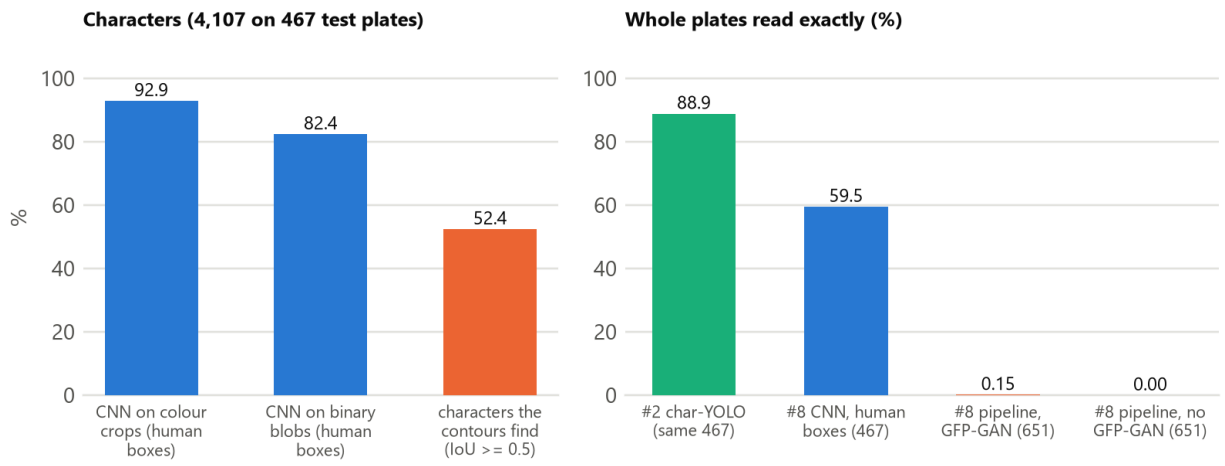


Figure 4. Left: the CNN reads 92.9 % of symbols cut by human boxes, 82.4 % when the same boxes are cut from the paper's black-and-white image, and the contours find only 52.4 % of the symbols at all. Right: whole plates read exactly. Paper #2's character detector (same v1.1 data) reads 88.9 % of the same 467 plates.

Step	What we measured	Loss
1. Cut the plate into symbols (Otsu + 15 largest contours)	a contour matches a symbol (IoU >= 0.5) for **52.4 %** of symbols; all symbols found on only 8.6 % of plates; 15.0 blobs per plate for 8.8 symbols	largest
2. Feed black-and-white blobs to a CNN trained on colour crops	92.9 % → 82.4 % with perfect boxes (letters 80.3 → 64.5, words 77.7 → 63.0)	-10.5 points
3. Classify the blobs that do match a symbol	93.5 % of 2,153 matched blobs right	small
4. Order the symbols	x order 41.9 % similarity, two-line order 49.5 %, our BRTA decoder 54.1 %; exact stays 0	cannot fix 1-2
5. Extra blobs	after the size filter the reader outputs 11.3 symbols per plate for 8.8 true ones; only 2.6 % of plates get the right count, 90.2 % too many	garbage symbols



Figure 5. What the contours see on three test plates (left: the crop; right: the paper's 333x335 Otsu image). Orange: the 15 largest contours; green dashed: the human symbol boxes. Contours lock onto the plate frame, holes inside letters, bolts and background; joined or broken Bangla words are missed.

Four real plates (similarity in brackets)

Truth	Paper recipe without GFP-GAN, x order	GFP-GAN + two-line order
খুলনা মেট্রো-ট ১১-১৬৩৬	৬৮১হ১খুলনা৮১থমেট্রো১৬১৩ট (0.67)	খুলনামেট্রোট১১১৬৩৬ (1.00)
ঢাকা মেট্রো-গ ৩৫-৫৩৭০	খুলনাজ৫৩৮৫মেট্রো৫ঢাকা৩৬৭০০গ (0.46)	৫০মেট্রো১৩৫৫৫৩৭০০ (0.71)
পিরোজপুর-হ ১১-৯১৬০	ঢাকাঢাকা২১১৫গ১রাজ৯ঢাকা১৬০০ (0.29)	৩গ০হ০১১৯১৬০ (0.54)
ঢাকা মেট্রো-গ ৩৭-৪৪৪৭	৫৫হ৬৫১ঢাকাগ০৫৮ঢাকা৫ঢাকাঢাকা (0.23)	ল (0.00)

Rows: the best plate (the only exact one), a plate at the 75th percentile, the median plate, and a bad night plate. Even the median plate scores about 0.5 similarity while being completely unreadable: similarity hides failure.

> **Bottom line for section 11.** Classic contour segmentation does not work on real Bangladeshi plates (two lines, joined Bangla words with মাত্রা, frames, bolts, tilt, glare, night). Even if segmentation were perfect, a per-symbol classifier reads 59.5 % of plates, while paper #2's character detector, trained on the same v1.1 data, reads 88.9 % of the same 467 plates: it is right on 144 plates where the CNN is wrong and wrong on only 7 where the CNN is right (McNemar $p < 0.0001$). A detector finds and names each symbol in one step, so there is no separate cutting step to fail.

12. GFP-GAN: does restoration help, and what does it cost?

This is the question the paper never answers. We ran the same 651 plates with and without the restoration step and the same CNN, and compared every plate with itself.

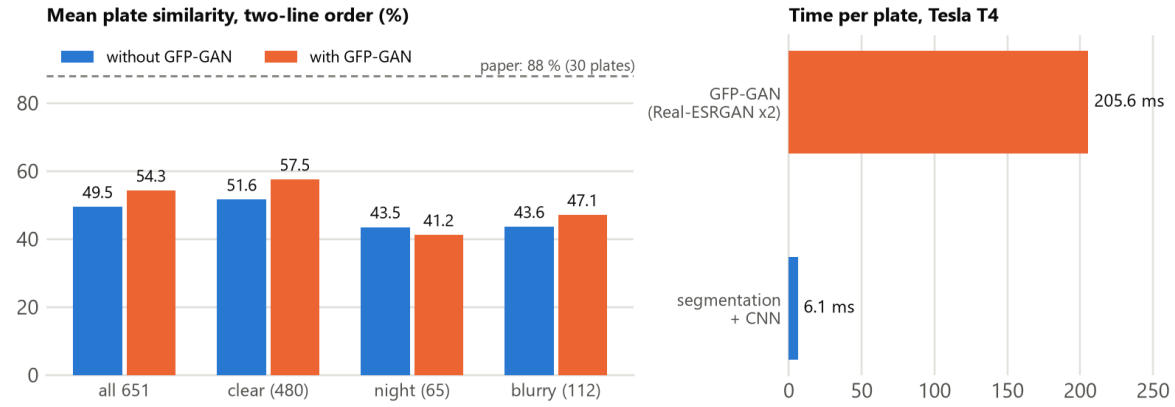


Figure 6. Left: mean similarity per condition, two-line order, with and without GFP-GAN (dashed: the paper's 88 % on 30 plates). Right: time per plate on a Tesla T4. GFP-GAN alone takes 34 times longer than segmentation and CNN together.

Paired result (651 plates)	Without	With GFP-GAN	Difference
Mean similarity, two-line order	49.49 %	54.30 %	+4.81 points; 343 plates better, 224 worse, 84 equal; Wilcoxon p = 4.2e-11
Mean similarity, x order (paper)	41.86 %	44.95 %	+3.09 points; p = 9.5e-08
Plates read exactly	0.00 % (0)	0.15 % (1)	+1 plate
Night (65 plates), similarity	43.51 %	41.22 %	-2.3 points (worse)
Blurry (112 plates), similarity	43.63 %	47.11 %	+3.5 points
Time per plate (T4)	6.1 ms	211.7 ms	35x slower in total; GFP-GAN alone allows at most 4.9 plates/s on a T4

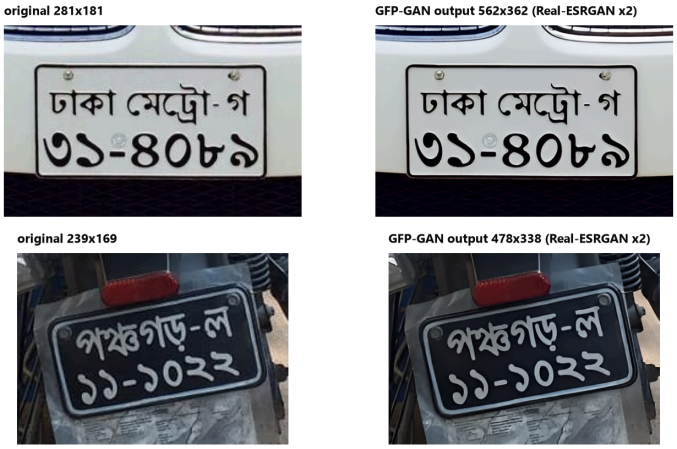


Figure 7. Two test plates before and after "GFP-GAN" (its Real-ESRGAN x2 step). Edges get sharper and the image doubles in size, but no stroke that is missing in the original comes back, and the Otsu and contour steps that follow stay the same.

> **Verdict.** Restoration gives a small, statistically real gain in partial similarity, mostly on clear and blurry daylight plates, but it does not turn unreadable plates into readable ones (1 extra exact plate in 651), it hurts at night, and it costs about 0.2 s per plate on a GPU, which alone breaks our 10 FPS laptop target (RQ3). The problem here is segmentation, which restoration does not fix. Any new upscaling test belongs in front of paper #2's detector, for small or blurry plates only (section 20).

13. Original-paper check: the paper's own data with our code

To be sure our port is right, we ran the kit's second notebook (`paper08_original_check.ipynb`, the same commands) locally on 24 Oct 2026 on an RTX 5060 Ti: the authors' `Data.zip` (pinned to commit 8dc72c7, checksum-verified), their split and settings, 3 seeds per model. The synthetic plate test of that notebook needs a Kaggle input and was not run.

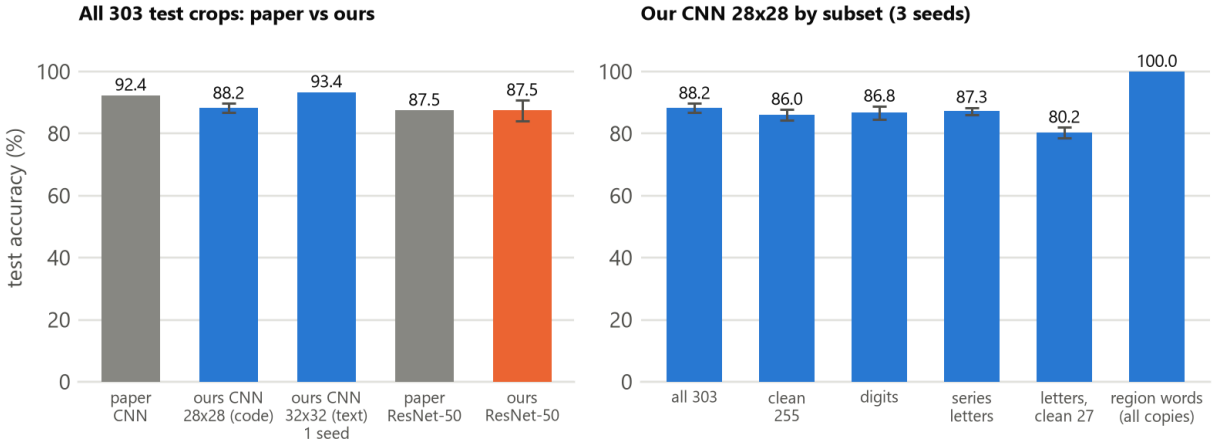


Figure 8. The authors' own test crops, last epoch as the paper reports (whiskers: spread of 3 seeds). Left: paper vs ours. Right: our CNN (28x28) by subset; "clean" = the 255 test crops that do not copy a training crop; all region-word test crops are copies.

On the authors' data	Paper (page)	Ours, 3 seeds
CNN test accuracy, last epoch	92.38 (p.4)	**88.23 ± 1.48**
CNN test accuracy, best-val checkpoint	- (authors' notebook: 93.40)	90.43 ± 0.93
CNN macro precision / recall / F1, last epoch	96.67 / 91.62 / 94.76 (p.4)	91.2 / 89.5 / 89.5
CNN best validation accuracy	98.80 (p.4)	98.29 ± 0.29
CNN on the 255 clean test crops, last epoch	not checked	86.01 ± 1.76
CNN digits / series letters / region words, last epoch	-	86.8 / 87.3 / 100.0
CNN series letters, clean crops only (27)	-	80.2 ± 1.8
CNN at 32x32 (the paper's text), 1 seed, last epoch	92.38 (p.3-4)	93.40
ResNet-50 test accuracy, last epoch	87.50 (p.4; notebook 73.93)	87.46 ± 3.27
ResNet-50 test accuracy, best-val checkpoint	-	91.64 ± 0.95
ResNet-50 with ImageNet normalisation (our fix), 1 seed, last / best-val	-	90.43 / 90.43
Training time per run (RTX 5060 Ti): CNN / ResNet-50	not reported	2.6 / 3.4 min

What the check shows. (1) **Our port is the paper's method.** ResNet-50 gives 87.5 ± 3.3 % (paper 87.50 %); the CNN at 32x32, the size the paper's text names, gives 93.4 % (one seed; paper 92.38 %); at 28x28, the size in the authors' code, 88.2 ± 1.5 %. So the code we ran on our data is not a broken copy. (2) **The leak inflates the score:** on the 255 clean test crops the CNN drops by 2.2 points (86.0 %) and on series letters by 7.0 points (87.3 → 80.2 %, 27 clean crops). The three region words score 100 %, but every one of their test crops copies a training crop: that 100 % is memory. (3) **Their validation set cannot choose a model:** 98.3 % validation accuracy, because 3 of 4 validation crops copy training crops. (4) **One seed cannot rank the models:** last-epoch accuracy moves by up to 8 points between seeds (ResNet-50 83.8 to 91.8 %), so Table II's single numbers (92.38 vs 87.50) do not show that the CNN is better.

On their small set (1,292 crops) the batch-1 CNN does not collapse: best epochs 31, 29, 30 of 40. This supports the explanation in section 8: the recipe is fine for 52,000 updates and unstable for 1.9 million. Our earlier CPU check (10 Oct, one seed, 28x28: 91.4 % last epoch) lies inside this spread. Kaggle run still to do: section 20.

14. Findings, and the numbers to quote

#	Finding	Number to quote	Section
1	The v1.1 run is healthy; the CNN recipe is unstable on 48k crops, so use the best-val model	5/5 jobs code=0; best epoch 5; local re-run identical	8
2	The symbol classifier works on common tokens	CNN 93.1 % of 4,107 symbols; 95.4 % on 21 common tokens	10
3	Rare tokens are the classifier's weak point	CNN 23.9 %, ResNet-50 45.5 % on 39 rare tokens	10
4	The paper's plate reader fails on real plates	0.15 % exact (1 of 651); 54.3 % similarity (paper 88 %)	9, 11
5	Segmentation is the cause	contours find 52.4 % of symbols; all of them on 8.6 % of plates	11
6	Per-symbol classification loses to a character detector even with perfect boxes	59.5 vs 88.9 % on the same 467 plates (144 vs 7, $p < 0.0001$)	11
7	GFP-GAN: small similarity gain, no real exact gain, worse at night, very slow	+4.8 points ($p = 4e-11$); +1 exact plate; 205.6 ms/plate	12
8	The paper reproduces on its own (leaky) data with our code	ResNet-50 87.5 % (paper 87.50); CNN 32x32 93.4 % (paper 92.38); 48/303 test crops copied	3, 13

What we keep and what we drop

From paper #8	Decision	Why
GFP-GAN / Real-ESRGAN restoration in front of the reader	**drop** from the main pipeline	finding 7; breaks RQ3 speed
Otsu + contour segmentation	**drop**	findings 4-6
Per-symbol CNN classifier	keep only as an idea for a second opinion on single symbols	finding 2; 760 FPS
17-class Bangla symbol crops (Data.zip)	do not mix into training (leaky, synthetic, 17 classes)	section 3
Similarity as the main metric	**replace** with plate exact + CER (similarity as an extra)	section 11: 0.5 similarity = unreadable
The restoration question	**keep** as an ablation (proposal: "x2 upscaling")	section 20

15. How to use this paper in our research

Our research part	How paper #8 is used	Evidence we now have (v1.1)
RQ2 / O3: character-level recognizer	the "segment, then classify" baseline that motivates a character *detector*	same 467 plates: per-symbol CNN with perfect boxes 59.5 % vs char-YOLO 88.9 %; real contours 0.15 %
RQ4 / G4: enhancement ablation ("CLAHE, x2 upscaling" in the proposal)	the first measured data point for x2 upscaling	+4.8 similarity points, +1 exact plate, night -2.3
RQ3 / O4: real time (>= 10 FPS)	restoration cost reference	GFP-GAN 205.6 ms per plate on a T4 = at most 4.9 plates/s
RQ1 / O1: duplicate-free split	a third leaky dataset next to KUET (#23) and BLPD (#2)	48 of 303 test and 320 of 429 val crops copied from train
O5: evaluation protocol	why we report plate exact, not only similarity	54.3 % mean similarity while 0.15 % exact
Method chapter (training)	why we keep the best-validation checkpoint and use early stopping	CNN last epoch 36.7 % vs best-val 93.1 %
Related work	the restoration-based Bangla system (already cited as [afrin2023] in the proposal)	paragraph below

16. What benefit do you get from it?

- **A clear answer to an open question.** "Does restoration help Bangla plate reading?" now has a measured, paired answer with a cost, which the paper itself does not give.
- **A negative result with a mechanism.** You can say *why* the pipeline fails (contours find half the symbols), not only that it fails. Examiners value that.
- **Evidence for our design.** The same-plates comparison with paper #2 (88.9 vs 59.5 %, $p < 0.0001$) justifies the character-detector design of our system with a number.
- **Proof that our code is right.** On the authors' data our port gives about their numbers; locally re-running the plate test gives the Kaggle numbers exactly.
- **A metric lesson for the whole team:** partial-credit similarity can look like 50 % on plates that are completely wrong.
- **A training lesson:** a batch-1 recipe that works on 1,292 crops collapses on 48,000; best-val checkpoints protect every kit run.
- **Reusable pieces:** a fast symbol classifier (760 FPS) and a token crop set of 47,992 training crops in 101 classes, made by the kit from our labels.

A paragraph you can adapt for the related-work chapter

```
Afrin et al. [ref] restore Bangla plate images with GFP-GAN, segment symbols with
Otsu thresholding and contours, and classify them with a small CNN, reporting 92.38%
symbol accuracy on 17 classes and 88% mean string similarity on 30 plates. Their
split is leaky (48 of 303 test crops copy a training crop) and no result without
restoration is given. Re-implemented on our 101-token benchmark, the classifier reads
93.1% of symbols, but contour segmentation finds only 52% of the characters, and the
full pipeline reads 0.15% of 651 real plates exactly. Restoration (on plates, GFP-GAN
reduces to Real-ESRGAN x2) adds 4.8 similarity points at 34 times the reading cost,
and even with perfect symbol boxes the per-symbol approach reads 59.5% of plates
versus 88.9% for a character detector on the same plates.
```

17. What to show the faculty (8 slides)

#	Slide title	What is on it	Source in this report
1	The paper and its claim	blurry plates; GFP-GAN + contours + CNN; 92.38 % symbols, 88 % similarity on 30 plates	sections 1, 5
2	The pipeline	the 6 steps, with our measured number under each	Figure 1
3	We reproduced it on their data	ResNet-50 87.5 % (paper 87.50), CNN 93.4 % (paper 92.38); but 48 of 303 test crops are copies	Figure 8, section 3
4	On our 101 tokens: the classifier is fine	93.1 % of symbols; rare tokens are the gap	Figure 3
5	Whole plates fail	0.15 % exact on 651 real plates; similarity hides it (example table)	section 9, 11
6	Why: segmentation	contours find 52.4 % of symbols; perfect boxes 59.5 % vs detector 88.9 %	Figures 4, 5
7	GFP-GAN: the missing ablation	+4.8 similarity points, +1 exact plate, worse at night, 34x the time	Figure 6
8	What we take for our system	drop restoration and contours; keep the character detector; report plate exact; upscaling as an ablation	section 14 table

> **Keep it honest:** call the 93 % "symbol accuracy on our split" and never compare it as a race with 92.38 % (different classes and test). Say "same plates" only for the comparison with paper #2. Do not show the CNN's last-epoch 36.7 % without its explanation (section 8). The original check was run on our own PC, not on Kaggle; say so if asked.

18. What to explain: a 3-minute script

"This 2023 ICCIT paper tries to read blurry Bangladeshi plates. It first sharpens the plate with GFP-GAN, then cuts it into symbols with a threshold and contours, and classifies each symbol with a small CNN. It reports 92 percent symbol accuracy and 88 percent similarity on 30 plates, but it never tests without GFP-GAN.

We first ran their code on their own data: we get their numbers, 87.5 percent for ResNet-50 and 93 percent for the CNN, so our port is correct. But 48 of their 303 test images are copies of training images, so their score is optimistic.

Then we trained the same models on our benchmark with 101 Bangla tokens. The classifier itself is good: 93 percent of symbols. But on 651 real plates the full pipeline reads only one plate exactly. We found why: the contour step finds only about half of the characters. It locks onto the plate frame, the holes inside letters and the bolts, and it breaks joined Bangla words. Even if we give it perfect human boxes, it reads 60 percent of plates, while our character detector from paper 2 reads 89 percent of the same plates.

Finally the missing test: with and without GFP-GAN on the same plates. GFP-GAN raises similarity by about 5 points, but it makes only one more plate fully correct, it is worse at night, and it takes 0.2 seconds per plate, 34 times the rest of the pipeline. So in our system we drop restoration and contour segmentation, keep the character detector, and report exact plate accuracy, not only similarity."

19. Questions the faculty may ask

Q1. Your CNN's last-epoch accuracy is 36.7 %. Did training fail?
No. The paper's recipe (batch 1, Adam, no normalisation) became unstable after epoch 5 on our 48,000 crops; one of our epochs equals the paper's whole training. We keep the best-validation checkpoint (93.1 % test), as every kit run does, and all plate results use it.

Q2. Is your 93 % better than their 92.38 %?
It is not a fair race: 101 tokens on a clean split vs 17 tokens on a leaky test. It shows the classifier idea works at our scale. On their own data our code gives their numbers: ResNet-50 87.5 ± 3.3 % (3 seeds; paper 87.50 %), CNN 93.4 % at 32x32 (paper 92.38 %).

Q3. They report 88 %, you get almost 0 %. Why so different?
Three reasons: (1) similarity vs exact (they never report exact plates; our similarity is 54.3 %); (2) their 30 test plates come from the clean synthetic set their symbols were cut from, ours are 651 real photos with night, blur and tilt; (3) their expected strings use the same x order as the reader, which hides line-mixing errors.

Q4. Maybe your implementation is wrong?
The same code reproduces their symbol accuracy on their data (section 13), the plate pipeline follows their notebook step by step (Figure 1), and a local re-run gives the Kaggle numbers exactly. The failure is measured step by step: the contours find 52.4 % of the symbols, and we can show it on images (Figure 5).

Q5. So does GFP-GAN help or not?
A little, on similarity: +4.8 points, 343 plates better and 224 worse, Wilcoxon $p < 10^{-10}$. Not on exact reading (1 extra plate), not at night (-2.3), and at 205.6 ms per plate it is too slow for real time.

Q6. You ran Real-ESRGAN, not GFP-GAN. Is that the same?
On a plate, yes. GFP-GAN v1.3 detects faces, restores them, and upsamples the rest of the image with Real-ESRGAN x2. A plate has no face, so the output is the Real-ESRGAN x2 image. We ran exactly that network (RealESRGAN_x2plus, the one the authors' settings load).

Q7. Why is macro F1 only 36 %?
Macro F1 averages over classes. 39 rare tokens have 1-19 test crops each and are mostly wrong; on the 21 common tokens macro F1 is 89.1 %. Rare districts are the real gap; the pretrained ResNet-50 does better on them.

Q8. Could better segmentation fix the pipeline?
Only partly. With perfect human boxes the per-symbol CNN still reads 59.5 % of plates vs 88.9 % for the character detector on the same plates. Learning to find and name symbols together works better.

Q9. Why compare with paper #2 at all?
Both were trained on the same v1.1 data and tested on the same plates by the same team code, so the comparison is paired and fair (McNemar: 144 plates only the detector reads, 7 only the CNN reads).

Q10. What will you do with restoration now?
Keep it as an ablation in our plan (x2 upscaling and CLAHE), but in front of the character detector and only for small or blurry plates, and report exact accuracy and milliseconds together.

20. Next steps (in order)

1. **Upload** (section 21) and fill the team table: `python add_results.py paper08_results.zip --member "Rokib Hasan Oli"` in `docs/results` (done for you on 24 Oct; check the notes' results block).
2. **Run** `paper08_original_check.ipynb` **on Kaggle** (about 1 h) with the input `furcifer/bangla-license-plates-synthetic`. It makes section 13 official and adds the one test we could not run: the plate test on the synthetic plates the authors cut their symbols from (the closest thing to their 30 plates).
3. **Upscaling ablation done right:** bicubic x2, Real-ESRGAN x2 and CLAHE in front of paper #2's character detector, on the small-plate and blurry subsets; report plate exact, CER and ms per plate.
4. **Rare tokens:** count district words with < 50 training crops and plan targeted data (or synthetic plates) for them; this limits every reader, not only #8.
5. **If a symbol classifier is ever reused** (e.g. as a second opinion for low-confidence district words): batch 64, learning-rate decay, early stopping, class-balanced sampling, ImageNet normalisation.
6. **Board and notes:** paste the code link and the one-line result (section 15 paragraph, short form) on the research board.

21. GitHub and Drive

`After Train/results.zip` (535 MB) is the whole Kaggle output; do not upload it. The upload folder is ready at `After Train/github_upload/weights/paper08/MD.Rokib.Hasan.Oli/` (same layout as paper #2 and #23):

File	What
`README.md`	results, model cards, quick start (tested), checksums
`best_cnn28.pt`	14.8 MB, the paper CNN trained on our 101 tokens (best-val epoch 5)
`best_resnet50_head.pt`	20.7 MB, the trained head of ResNet-50. The full best.pt is 115 MB (over GitHub's limits), but its backbone is frozen ImageNet ResNet-50, checked bit-identical, so head + torchvision rebuild the exact model (README)
`paper08_results.zip`	0.14 MB, all results, logs, per-plate predictions and code, no weights
`results_vs_paper.md`	the kit's comparison table
`Paper08_Study_Report.pdf`	this report
`local_checks.json`	the numbers of sections 11 and 13 (local checks)

Drive `498R / weights / paper08 /`: `After Train/for_drive/paper08_outputs.zip` (230 MiB: both models with best.pt and last.pt, full size). Web upload on GitHub: open `weights` → Add file → Upload files → drag the `paper08` folder → commit message "Add paper #8 results on dataset v1.1 (Rokib)". Every file is under 25 MB.

Appendix: where every number comes from

Number	Source
paper numbers	arXiv 2312.10701, pages given in the text; paper-vs-code facts from the notes (repository commit 8dc72c7)
v1.1 training, classifier, plate and speed results	`After Train/results.zip` → `runs/*/results.json`, `log.csv`, `per_plate.csv`; `paper-8.log`
token groups, rare classes, confusions, paired GFP-GAN test, counts per plate	`report_src/analysis.py` → `analysis.json`
segmentation diagnosis, binary-blob test, BRTA decoder, same plates vs paper #2	`report_src/seg_check.py` → `seg_check.json` (CPU, the run's best.pt, dataset v1.1; paper #2 predictions from its v1.1 results.zip)
original-paper check	`report_src/check_collect.py` → `check_local.json` (local run of the kit commands, RTX 5060 Ti, 24 Oct 2026)
upload weights	`report_src/make_upload_weights.py` (backbone check, head file)

```
@inproceedings{afrin2023bengali,
  title     = {Bengali License Plate Recognition: Unveiling Clarity
               with {CNN} and {GFP-GAN}},
  author    = {Afrin, Noushin and Hasan, Md Mahamudul and Safin, Mohammed
               Fazlay Elahi and Amin, Khondakar Rifat and Haque, Md Zahidul
               and Ahmed, Farzad and Shawon, Md Tanvir Rouf},
  booktitle = {2023 26th International Conference on Computer and
               Information Technology (ICCIT)},
  address   = {Cox's Bazar, Bangladesh},
  year      = {2023},
  note      = {arXiv:2312.10701}
}
```